

```

0000 compiler-- Copyright (C) 2024-2025 Vincent MORIN - Universit  de Bretagne Occidentale-- This program is free software: you can redistribute it and/or
0001 or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or
0002 (at your option) any later version. This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
0003 the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this program. If not, see https://www.gnu.org/licenses/.
0004 with UNCHECKED_CONVERSION; separate (IDL);
0005 IDL MAN.ADB-Vincent MORIN
0006 1-2-3-4-5-6-7-8-9-A-B-C
0007 package body IDL_MAN is
0008
0009 --FUNCTION ARITY--function ARITY ( T : TREE ) return ARITIES is begin return N_SPEC ( T.TY ).NS_ARY; return
0010 ER L'ARITE SUIVANT LES SPECIFS DU TYPE DE NOEUD ARBRE;end;
0011 --FUNCTION SON_1--function SON_1 ( T : TREE ) return TREE is begin if N_SPEC ( T.TY ).NS_ARY in UNARY then return SI L'ARITE LE PERMET
0012 return DABS ( 1, T.V ); return LE PREMIER SOUS-ARBRE DE T; end if; PUT_LINE ( "IDL.IDL_MAN.SON_1 : PAS DE FILS 1 LI
0013 SIBLE POUR " & NODE_REP ( T ) ); raise PROGRAM_ERROR;end;
0014 --PROCEDURE SON_1--procedure SON_1 ( T : TREE; V : TREE ) is begin if N_SPEC ( T.TY ).NS_ARY in UNARY then return SI L'ARITE LE PERMET
0015 DABS ( 1, T.V ); STOCKER V COMME-PREMIER SOUS ARBRE DE T; else PUT_LINE ( "IDL.IDL_MAN.SON_1 : PAS DE FILS
0016 INSCRIPTIBLE POUR " & NODE_REP ( T ) ); raise PROGRAM_ERROR; end if;end;
0017 --FUNCTION SON_2--function SON_2 ( T : TREE ) return TREE is begin if N_SPEC ( T.TY ).NS_ARY in BINARY then return NARY then
0018 return DABS ( 2, T ); end if; PUT_LINE ( "IDL.IDL_MAN.SON_2 : PAS DE FILS 2 LISIBLE POUR " & NODE_REP ( T ) ); raise PROGRAM_ERRO
0019 R;end;
0020 --PROCEDURE SON_2--procedure SON_2 ( T : TREE; V : TREE ) is begin if N_SPEC ( T.TY ).NS_ARY in BINARY then DABS ( 2, T.V ); else PUT_LINE ( "IDL.IDL_MAN.SON_2 :
0021 PAS DE FILS 2 INSCRIPTIBLE POUR " & NODE_REP ( T ) ); raise PROGRAM_ERROR; end if;end;
0022 --FUNCTION SON_3--function SON_3 ( T : TREE ) return TREE is begin if N_SPEC ( T.TY ).NS_ARY = TE
0023 RARY then return DABS ( 3, T ); end if; PUT_LINE ( "IDL.IDL_MAN.SON_3 : PAS DE FILS 3 LISIBLE POUR " & NODE_REP ( T ) ); raise PROGRAM_ERROR;
0024 end;
0025 --PROCEDURE SON_3--procedure SON_3 ( T : TREE; V : TREE ) is begin if N_SPEC ( T.TY ).NS_ARY = TERNARY then DABS ( 3, T.V ); else PUT_LINE ( "IDL.IDL_MAN.SON_3 : PAS DE FILS 3
0026 INSCRIPTIBLE POUR " & NODE_REP ( T ) ); raise PROGRAM_ERROR; end if;end;
0027 --FUNCTION HEAD--function HEAD ( S : SEQ_TYPE ) return TREE is begin if S.FIRST.TY = DN_LIST then return LA
0028 SEQ CONTIENT-UNE LISTE; return DABS ( 1, S.FIRST ); return LE PREMIER CHAMP DU NOEUD LISTE; elsif S.FIRST = TREE_NIL then return LA
0029 SEQ CONTIENT-UN ELEMENT; return S.FIRST; return RENDRE CELUI-CI; end if; PUT_LINE ( "IDL.IDL_MAN.HEAD : TETE DE SEQUENCE SEQ.FIRST = TREE_
0030 NIL " ); raise PROGRAM_ERROR;end;
0031 --FUNCTION TAIL--function TAIL ( S : SEQ_TYPE ) return SEQ_TYPE is begin return UN SEQ-SUITE DE LISTE; begin if S.FIRST.TY = DN_LIST then return LI
0032 ON TAIL; function TAIL ( S : SEQ_TYPE ) return SEQ_TYPE is begin return UN SEQ-SUITE DE LISTE; begin if S.FIRST.TY = DN_LIST then return LI
0033 STE-A PLUSIEURS ELEMENTS; declare QUEUE : SEQ_TYPE := ( FIRST=>DABS ( 2, S.FIRST ), NEXT=>TREE_NIL ); TETE DE QUEUE ET RIEN; begi
0034 n; if QUEUE.FIRST.TY = DN_LIST then return SI TETE INDIQUANT UNE LISTE AVEC UNE SUITE; QUEUE.NEXT := S.NEXT; return LA SUITE DE LA
0035 QUEUE EST RENDUE; end if; return QUEUE; end; elsif S.FIRST = TREE_NIL then return LISTE-A UN SEUL-ELEMENT; return ( TREE_NIL,
0036 TREE_NIL ); return RETOURNER UN SEQ VIDE; end if; PUT_LINE ( "IDL.IDL_MAN.TAIL : LISTE VIDE S.FIRST = TREE_NIL " ); raise PROGRAM_ERROR;end
0037 TAIL;
0038 --FUNCTION INSERT--function INSERT ( S : SEQ_TYPE; T : TREE ) return SEQ_TYPE is begin INSERTION D'UN SEQ EN TETE DE SEQUENCE; begin if S.FIRST = TREE_NIL then return SEQUENCE VIDE;
0039 return ( FIRST=>T, NEXT=>TREE_NIL ); return RETOURNER UN SEQ A UN ELEMENT; end if; declare LE SEQ INITIAL ETAIT NON VIDE;
0040 T_SEQ := SEQ_TYPE := ( FIRST=>MAKE ( DN_LIST ), NEXT=>S.NEXT ); NOUVELLE SEQUENCE; begin DABS ( 1, T_SEQ.FIRST, T ); TETE DE SE
0041 QUENCE SUR T; DABS ( 2, T_SEQ.FIRST, S.FIRST ); SUITE-CONSTITUEE PAR LE DEBUT DE S; if S.NEXT = TREE_NIL then return SI S NE CONT
0042 IENT QU'UN ELEMENT; T_SEQ.NEXT := T_SEQ.FIRST; LA SUITE EST CONFONDUE AVEC LE PREMIER ELEMENT; end if; return T_SEQ; end;end;
0043 --FUNCTION APPEND--function APPEND ( S : SEQ_TYPE; T : TREE ) return SEQ_TYPE is begin T_SEQ := SEQ_TYPE; begin if S.FIRST = TREE_NIL then return LISTE-VIDE; return ( FIRST=>T, NEXT=>TR
0044 EE_NIL ); return SEQUENCE T EN TETE RIEN DERRIERE; elsif S.FIRST.TY = DN_LIST then return SEQUENCE A UN ELEMENT (PAS-DE LISTE EN S.FIRST);
0045 T_SEQ.FIRST := MAKE ( DN_LIST ); FABRIQUER UNE LISTE; DABS ( 1, T_SEQ.FIRST, S.FIRST ); L'ELEMENT DE S EST EN TETE; DABS ( 2
0046 , T_SEQ.FIRST, T ); T SUIT EN FIN; T_SEQ.NEXT := T_SEQ.FIRST; SEQUENCE TETE ET SUITE CONFONDUES; else EST-UNE
0047 LISTE-A PLUS D'UN ELEMENT; declare T_TAIL : TREE := S.NEXT; T_END : TREE; begin if S.NEXT = TREE_NIL then return LA SEQUE
0048 NCE S N'A QU'UNE TETE; T_TAIL := S.FIRST; LA QUEUE EST LE-DEBUT; end if; loop T_END := DABS ( 2, T_TAIL );
0049
0050 -- TREE DE FIN DE S; exit when T_END.TY /= DN_LIST; SORTIE EN FIN DE LISTE (SIMPLE POINTEUR A UN ELEMENT); T_TAIL := T_END;
0051 SUIVRE LA LISTE; end-loop; T_SEQ.FIRST := S.FIRST; T_SEQ.NEXT := MAKE ( DN_LIST ); FABRIQUER UN ELEMENT DE LISTE;
0052 DABS ( 1, T_SEQ.NEXT, T_END ); TETE DE LISTE; DABS ( 2, T_SEQ.NEXT, T ); QUEUE-DE LISTE; DABS ( 2, T_TAIL, T_SEQ.N
0053 EXT ); CHAINAGE; end; end if; return T_SEQ;end;
0054 --FUNCTION SINGLETON--function SINGLETON ( T : TREE ) return SEQ_TYPE is begin return ( FIRST=>T, NEXT=>TREE_NIL );end;
0055 --PROCEDURE LIST--procedure LIST ( T : TREE; S : SEQ_TYPE ) is A_IDX : INTEGER := N_SPEC ( T.TY ).NS_FIRST; for I in 1 .. N_SPEC ( T.TY ).NS_SIZE loop parcourir LES ATTRIBUTS
0056 if A_SPEC ( A_IDX ).IS_LIST then return SI ATTRIBUT LISTE RENCONTRE; DABS ( 1, T, S.FIRST ); STOCKE LA TETE DE V DANS L'ATTRIBUT
0057 I DU NOEUD POINTE PAR T; return C'EST-BON, SORTIR; end if; A_IDX := A_IDX + 1; ATTIBUT SUIVANT; end loop; PUT_
0058 LINE ( "IDL.IDL_MAN.LIST : PAS DE LISTE INSCRIPTIBLE DANS " & NODE_REP ( T ) ); raise PROGRAM_ERROR;end;
0059 --PROCEDURE DABS--procedure DABS ( RANG : ATTR_NBR; T : TREE; VAL : TREE ) is RN : RP
0060 G_IDX; begin if T.PG => CUR_VP then LA PAGE QUI NOUS INTERESSE N'EST PAS COURANTE; CUR_VP := T.PG; LA MENTIONNER COMME COU
0061 RANTE; RN := ASSOC_PAGE ( CUR_VP ); SON ASSOCIEE PHYSIQUE EST LA RN; IF RN = 0 OR ELSE PAG ( RN ).RECUPERABLE THEN return SI ELLE ES
0062 T FLOTTANTE; if RN = 0 then return SI HORS MEMOIRE; CUR_RP := READ_PAGE ( CUR_VP ); ASSURER LA PAGE-PHYSIQUE; else
0063 -- NON FLOTTANTE; CUR_RP := RN; PAGE REELLE COURANTE; end if; end if; PAG ( CUR_RP ).DATA.all ( T.LN + RANG ) := VAL;
0064 ECRIRE; PAG ( CUR_RP ).CHANGED := TRUE; MENTIONNEE CHANGE ( ON Y A ECRIT );end;
0065 --FUNCTION DABS--function DABS ( RANG : ATTR_NBR; T : TREE ) return TREE is RN : RP; PG_IDX; begin if
0066 f T.PG => CUR_VP then LA PAGE DE T N'EST PAS LA-COURANTE; CUR_VP := T.PG; LA MENTIONNER COMME COURANTE; RN := ASSOC_PAGE (
0067 CUR_VP ); PAGE REELLE ASSOCIEE : RN; IF RN = 0 OR ELSE PAG ( RN ).RECUPERABLE THEN return SI ELLE EST FLOTTANTE; if RN = 0 then
0068 -- SI HORS MEMOIRE; CUR_RP := READ_PAGE ( CUR_VP ); ASSURER LA PAGE-PHYSIQUE; else NON FLOTTANTE; CUR_RP := R
0069 N; PAGE REELLE COURANTE; end if; end if; return PAG ( CUR_RP ).DATA.all ( T.LN + RANG );end;
0070 --FUNCTION STORE TEXT--function STORE TEXT ( S : STRING ) return TREE is
0071 -- STOCKE UNE REPRESENTATION-TEXTE; NB TREES : LINE_IDX := LINE_IDX ( ( S'LENGTH+1 ) * CHARACTER'SIZE + TREE'SIZE ) / TREE'SIZE; -- NO
0072 MBRE DE TREES-POUR CONTENIR LES CARACTERES DE S ET UN-OCTET DE LONGUEUR NB_CHARS := NATURAL ( NB TREES ) * TREE'SIZE / CHARACTER'SIZE; be
0073 gin declare TTR : TREE := MAKE ( DN_TXTREP, NB_ATTR=>NB TREES, AR=>9 ); FABRIQUER LE NOEUD CONTENANT LE TEXTE; type TTREES is array (
0074 1..NB TREES ) of TREE; subtype LSTR is STRING ( 1..NB_CHARS ); function TO TREES is new UNCHECKED_CONVERSION ( LSTR, TTREES ); A_COPIER : LS
0075 TR := ( others=>ASCII.NUL ); START : LINE_IDX := T.LN; TTR := TTREES; begin A_COPIER ( 1..S'LENGTH+1 ) := CHARACTER'VAL ( S'LENGTH
0076 + S ); TTR := TO TREES ( A_COPIER ); for I in 1..NB TREES loop PAG ( CUR_RP ).DATA.all ( START+I ) := TTR ( I ); end loop; PAG ( CUR
0077 RP ).CHANGED := TRUE; MENTIONNEE CHANGE ( ON Y A ECRIT ); return TTR;end;
0078 --FUNCTION HASH_SEARCH--function HASH_SEARCH ( S : STRING ) return TREE is NB TREES : ATTR_NBR;
0079 -- NOMBRE DE TREES-POUR CONTENIR LES CARACTERES DE S ET UN-OCTET DE LONG
0080 UEUR; NB_CHARS := NATURAL ( NB TREES ) * TREE'SIZE / CHARACTER'SIZE; type TTREES is array ( 1..NB TREES ) of TREE; subtype LSTR is
0081 STRING ( 1..NB_CHARS ); CHAINE DE S AVEC UN OCTET-DE LONGUEUR function TO TREES is new UNCHECKED_CONVERSION ( LSTR, TTREES ); CONVE
0082 RSION DE LA CHAINE EN TABLEAU DE TREES; TTR := TTREES; VARIABLE CONVERTIE; A_COPIER : LSTR := ( others=>ASCII.NUL ); HASH_SUM : INTE
0083 GER := 0; VALEUR DE HACHAGE; function TO INT is new UNCHECKED_CONVERSION ( TREE, INTEGER ); begin A_COPIER ( 1..S'LENGTH+1 ) := CHARACTE
0084 R'VAL ( S'LENGTH ) & S; TTR := TO TREES ( A_COPIER ); VARIABLE CONVERTIE; for I in 1..NB TREES loop BOULE DE CALCUL DE LA VA
0085 LEUR DE HACHAGE; HASH_SUM := abs ( HASH_SUM ) TO INT ( TTR ( I ) ); end loop; declare BUCKET : LINE_IDX := LINE_IDX ( HASH_SUM mod INTEGER (
0086 LINE_IDX'LAST ) + 1 ); SEAU DE HACHAGE-INDICE DANS LE BLOC DES TETES DE LISTE; HASH_LIST : SEQ_TYPE := ( FIRST=>DABS ( BUCKET, TREE_HASH )
0087 , NEXT=>TREE_NIL ); TETE DE LISTE HACHEE; begin while HASH_LIST.FIRST = TREE_NIL loop SUIVRE LA LISTE-DU SEAU; declare
0088 SYM_T : TREE := HEAD ( HASH_LIST ); POINTEUR SYMBOL REP; TXT_T : TREE := DABS ( 1, SYM_T ); LE POINTEUR TXTREP AU NOEUD CONTENIA
0089 NT LE TEXTE; begin if DABS ( 0, TXT_T ).NSIZ = NB TREES then SI LA-LONGUEUR DU BLOC CORRESPOND A-CELLE DU CONVERTI; declare IS
0090 _MATCH : BOOLEAN := TRUE; SUPPOSER QUE CELA VA MARCHER; START : LINE_IDX := TXT_T.LN; INDICE DE L'ENTREE DU BLOC CONTENANT LE TEXTE
0091 -A TESTER; RP_RPG : RP; DATA renames PAG ( CUR_RP ); LA PAGE CONCERNEE; begin for I in 1..NB TREES loop POUR TOUS LES TREES CO
0092 UVRANT LE TEXTE; if TTR ( I ) /= RP_RPG.DATA.all ( START+I ) then DEUX MORCEAUX DE TEXTE DIFFERENTS; IS_MATCH := FALSE; DESAC
0093 CORDER; exit; end if; end loop; if IS_MATCH then ACCORDER; return SYM; return RETOURNER LE SYMBOLE TROUVE; end if
0094 ;end; end if; HASH_LIST := TAIL ( HASH_LIST ); CONTINUER SUR LE RESTE DE LA LISTE; end; end loop; return ( HI,
0095 NOTY=>DN_NUM_VAL, ABS=>0, NSIZ=>BUCKET ); return LE SEAU;end;
0096 --FUNCTION STORE SYM--function STORE SYM ( S : STRING ) return TREE is INSERTER UN SYMBOLE A
```

[illegible]